
Deep Learning Project Report

Quentin Goulas, Lucas Hocquette

ICE P24

Table des matières

1	Project introduction	2
2	Algorithm descriptions	3
2.1	Grid search	3
2.2	Random search	3
2.3	Particle Swarm Optimization	4
2.4	Genetic Algorithms	4
2.5	Bayesian Optimization HyperBand algorithm [falknerBOHBRobustEfficient2018]	5
3	Development framework	6
4	Experimental setup	6
5	Results	7
	References	7

This project report aims to present the development framework, methodology and results obtained during the Deep Learning course project. This project focussed on hyperparameter optimization methods for deep learning models.

This report comprises four sections : an introduction to the project its premises, a presentation of the considered algorithms for this project, a description of the deployed system architecture and finally a discussion on the obtained results.

The source code for this project can be found at this repository [\[link\]](#)

1 Project introduction

In the context of deep learning, hyperparameters are model parameters that aren't learnt by the model yet impact the learning capacity of the model. Hyperparameters include, but are not limited to :

- model architecture : number of hidden layers, size of the hidden layers, layer type (dense, convolutional, recurrent, ...), activation function of the layer
- preprocessing : normalization, data augmentation, ...
- regularization : Lasso, Tykhonov, dropout layers, ...
- loss function choice : euclidean distance, cross-entropy loss, ...

The choice of good hyperparameters is crucial to have a well-functioning model where having too simple a model may induce underfitting whilst building too complex a model might lead to overfitting. Yet, choosing the appropriate hyperparameters implies training an important number of models with different hyperparameters configuration, which induces high computing costs. Finding efficient methods to obtain the optimal hyperparameter configuration for a given DL task is therefore a topic of choice for the deep learning literature.

The most used Hyper Parameter Optimization (HPO) methods include [\[yangHyperparameterOptimization\]](#)

- Babysitting : try different hyperparameter configurations, assess the accuracy of the different models, manually adjust the configurations and repeat till convergence. This method can be computationally intensive and labor intensive
- Grid search : try all the possible hyperparameter configurations and select the best one. This method requires very important computational and time resources, as well as a discrete hyperparameter space to evaluate
- Random search : try a proportion of randomly chosen hyperparameters in the possible hyperparameter space and select the best performing configuration among the tested configuration
- Gradient-based optimization : uses a gradient-descent approach to find the next hyperparameter configuration to evaluate. This requires a loss function that is differentiable in the hyperparameter variables
- Particle Swarm Optimization (PSO) : uses the particle swarm optimization algorithm to find the optimum for a given function
- Genetic Algorithms which uses the genetic heuristic to find the optimal hyperparameters
- Bayesian methods using a modelled prior on the model loss function to converge to the best hyperparameter configuration

This project aims at implementing a HyperParameter Optimization framework to compare different hyperparameter optimizers. For this project, we will focus on Grid Search, Random Search, Bayesian Optimizer Hyper Band and Particle Swarm Optimization (said to be the best hyperparameter optimizing method in the current literature)

2 Algorithm descriptions

We describe here the algorithms that have been implemented throughout this project : Grid search, Random search, Particle swarm optimization. We also provide the pseudo-codes for the methods.

2.1 Grid search

This method consists in “brute forcing” the best hyperparameter configuration, i.e by trying all possible hyperparameter configurations on the hyperparameter space and returning the best configuration. This method is extremely computationnaly heavy, with the number of possible configurations increasing exponentially as the number of tested hyperparameters increases.

Algorithm 1: Grid search algorithm

Input: Hyperparameter space H , training set D , validation set V
Output: Best hyperparameter configuration h^* and accuracy a^*
for each hyperparameter configuration h in H **do**
 Train model M with hyperparameters h on D ;
 Evaluate model M on V and compute accuracy a ;
 if $a > a^*$ **then**
 $h^* \leftarrow h$;
 $a^* \leftarrow a$;
 end
end
return h^*, a^*

2.2 Random search

This method consists in randomly selecting a proportion of all the possible hyperparameter configurations and returning the best configuration after trying all configurations in the sample. This method reduces slightly the computational time compared to grid search. However, this method does not solve the curse of dimensionality problem. As a matter of fact, the only method to solve this problem is to use a sampling proportion of the order p^d with the d the number of hyperparameters. This technique keeps computation time constant but dramatically reduces the exhaustivity of the method and success probability of the algorithm.

Algorithm 2: Random search algorithm

Input: Hyperparameter space H , training set D , validation set V , sampling proportion p
Output: Best hyperparameter configuration h^* and accuracy a^*
for each hyperparameter configuration h in H **do**
 $r \sim \mathcal{U}([0, 1])$;
 if random number $r < p$ **then**
 Train model M with hyperparameters h on D ;
 Evaluate model M on V and compute accuracy a ;
 if $a > a^*$ **then**
 $h^* \leftarrow h$;
 $a^* \leftarrow a$;
 end
 end
end
return h^*, a^*

2.3 Particle Swarm Optimization

The particle swarm optimization algorithm is a metaheuristic optimization algorithm which assumes next to no information on the functions to optimize. Modelled on the evolution of bird flocks, it consists in evaluating the model accuracy on a small set of hyperparameter configurations, then choosing a new set of configurations on the basis of the best configuration seen overall and by the best configuration seen by the particles individually.

Algorithm 3: Particle Swarm Optimization (PSO) algorithm

Input: Hyperparameter space H , training set D , validation set V , swarm size S , ϕ_l local step size, ϕ_g global step size, w inertia, precision ϵ

Output: Best hyperparameter configuration h^* and accuracy a^*

Initialize the personal best position vector $\mathbf{p}_0$ to $\mathbf{x}_0$;

Initialize the position vector (randomly taken from H) $\mathbf{x}_0$ of the swarm;

Initialize the speed vector $\mathbf{v}_0$ to $\mathbf{0}$;

Initialize and compute accuracy vector $\mathbf{a}_0$ of the swarm $\mathbf{x}_0$;

$t \leftarrow 0$;

$a^*, h^* \leftarrow \max(\mathbf{a}_0), \mathbf{x}_0[\text{argmax}(\mathbf{a}_0)]$;

repeat

 Compute $\mathbf{a}_t$ of the swarm $\mathbf{x}_t$;

$\tilde{a}, \tilde{h} \leftarrow \max(\mathbf{a}_t), \mathbf{x}_t[\text{argmax}(\mathbf{a}_t)]$;

if $\tilde{a} > a^*$ **then**

$a^*, h^* \leftarrow \tilde{a}, \tilde{h}$;

end

$t \leftarrow t + 1$;

 Take u_l and u_g uniformly from $[0, \phi_l]$ and $[0, \phi_g]$;

$\mathbf{v}_t \leftarrow \mathbf{v}_{t-1}.w + (\tilde{h} - \mathbf{x}_{t-1}).u_l + (h^* - \mathbf{x}_{t-1}).u_g$;

$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} + \mathbf{v}_t$;

until $\|\mathbf{v}_t\| < \epsilon$;

return h^*, a^*, t

The convergence of the Particle Swarm Optimization algorithm cannot be proven in the general case (although some refinements can provide convergence to a local optimum [engelbrechtConvergenceProofPar]). Yet, empirical data shows the efficiency of this algorithm in a majority of cases.

2.4 Genetic Algorithms

The genetic algorithm is a metaheuristic optimization algorithm inspired by the process of natural selection. It uses the concepts of selection, crossover and mutation to evolve a population of individuals towards the best solution. The algorithm starts by generating an initial population of individuals, then iteratively selects the best individuals, crosses them over to create new individuals and mutates them to introduce diversity in the population. The algorithm stops when a stopping criterion is met, such as a maximum number of generations or a convergence criterion. In our case each individual represents a hyperparameter configuration and the fitness of an individual is the accuracy of the model trained with this configuration. It is a similar algorithm to PSO but with a different methodology to evolve the population. The pseudo-code of a genetic algorithm is given below.

Algorithm 4: Genetic Algorithm (GA) algorithm

Input: Hyperparameter space H , training set D , validation set V , population size S , n_b number of best individual selected, n_r number of random individual selected, mutation rate p_m , crossover rate p_c , number of generations G

Output: Best hyperparameter configuration h^* and accuracy a^*

Initialize the population of size S with random hyperparameter configurations;

for $g \leftarrow 1$ **to** G **do**

 Evaluate the fitness (accuracy) of each individual in the population;

 Select n_b best individuals and n_r random individuals to form the parents of the next generation;

 Apply crossover with probability p_c to create new individuals;

 Apply mutation with probability p_m to introduce diversity in the population;

end

return *Best individual in the population (best hyperparameter configuration and accuracy)*

Selection step

The selection step is used to select the best individuals in the population to form the next generation. There are several methods to perform the selection step, but here we choose a number n_b of the best individuals in the population based on their accuracy and a number n_r of random individuals to form the parents of the next generation. The random individuals are chosen in order to keep diversity in the population and not fall into a local optimum.

Crossover step

The crossover step is used to create new individuals from the parents. In our case, we use a uniform crossover method where two parents give two children by exchanging their hyperparameters with a certain probability p_c . This assure a local exploration of the hyperparameter space by similar and accurate individuals but also a more global exploration by the random individuals.

Mutation step

The mutation step is used to introduce diversity in the population. It consists in choosing with a probability p_m a hyperparameter of an individual and changing it to a random value in the hyperparameter space. This assures that we theoretically explore the whole hyperparameter space if we compute enough generations but also that we introduce randomized diversity in the population.

One must carefully choose the values of S , n_b , n_r , p_m and p_c in order to assure a fast convergence to a good solution. The algorithm proposed and implemented in this project is a simple genetic algorithm, but there are many variations of genetic algorithms that can be very useful in the case of hyperparameter optimization.

2.5 Bayesian Optimization HyperBand algorithm [falknerBOHBRobustEfficient2018]

This method uses a probabilistic model of the accuracy function given the different tested configurations and iteratively returns the next configuration to try out by choosing the maximizer of expected improvement. The probabilistic model used here is the Tree Parzen Estimator which uses a kernel density estimator to model the accuracy function of the model. To further improve the performance of the algorithm, the Hyper Band technique is used to dynamically allocate the amount of resources for each training. For this project, we rely on the HpBandSter library [clickable link] which provides a full BOHB optimizer framework —similar to ours— that we integrated to our framework. Furthermore, this library enables asynchronous processing to try out multiple configurations simultaneously, therefore drastically increasing computing efficiency.

3 Development framework

This project will be entirely developed on Python through the Pytorch deep learning library. Moreover, the CIFAR-10 dataset will be used throughout this study to train a LeNet5-type model architecture for image recognition. Our hyperparameters of interest are the output sizes of the convolutional layers (C1, C3, C5) and the output size of the first dense layer (F6)

For this project, we decide to build a HyperParameterOptimizer (HPO) object responsible for navigating the input hyperparameter space and organizing the different hyperparameter configuration tests. The HPO object then calls a dedicated library to generate a model with the appropriate hyperparameters and launches the trainings. The system architecture is given in Figure 1.

The accuracy function of the classifier is chosen as the proportion of accurate classifications from the model on a validation dataset.

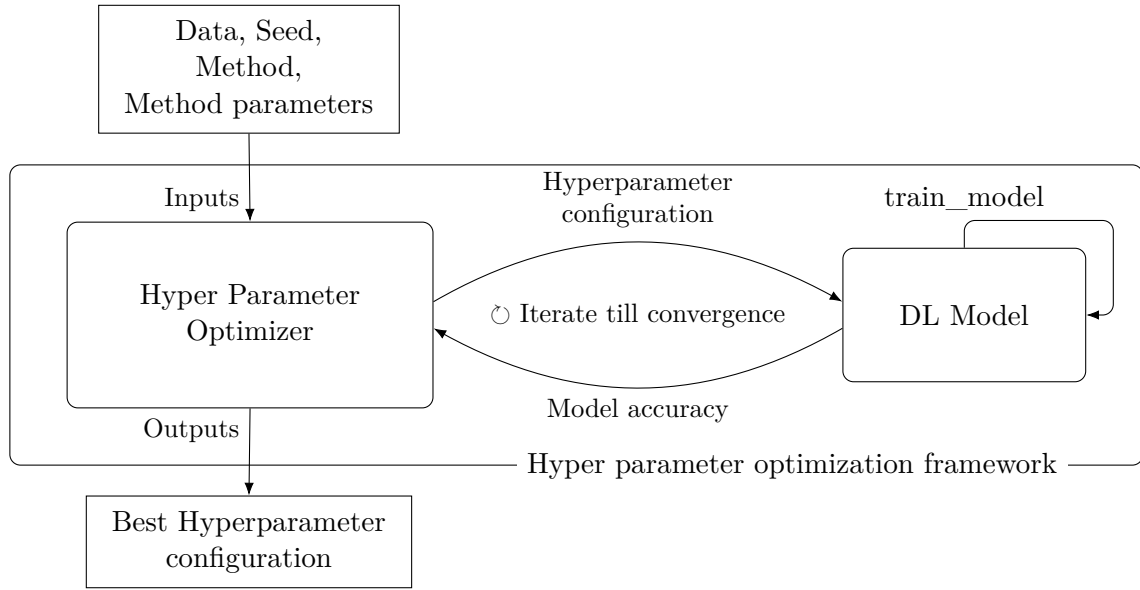


FIGURE 1 – Hyper parameter optimization workflow

To improve scalability, hyperparameter configurations are parsed as dictionaries, with name-value pairs of hyperparameter name and corresponding values. This enables to manipulate the same objects across hyperparameter spaces (which are hyperparameter configurations with multiple accessible values) and hyperparameter arrays (used to represent multiple hyperparameter configurations simultaneously).

4 Experimental setup

To try out the different hyperparameter optimizers, we search for the optimal configurations of a LeNet-5 type network [lecunGradientbasedLearningApplied1998]. More precisely, we look for the optimal number of filters for the three convolutional layers C1, C3 and C5 as well as the size of the network's dense layer F6. A schematic of the LeNet-5 architecture is provided in Figure 2.

We use the LeNet-5 architecture to recognize different images from the CIFAR-10 dataset [krizhevskyLearning divided into 10 categories : { airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck}. The CIFAR-10 comprises 60 000 coloured images of 32×32 pixels with 6 000 images per category and split into a 50 000 image training set and a 10 000 image test set.

Model trainings for this project have been entirely done on local GPUs (GeForce MX230 and GeForce RTX3070 Laptop) with a batch of size 64. Training the classifier on CIFAR-10 as provided by torchvision led to an epoch duration from 20 to 40s depending on model complexity.

We use the following parameters for each algorithm :

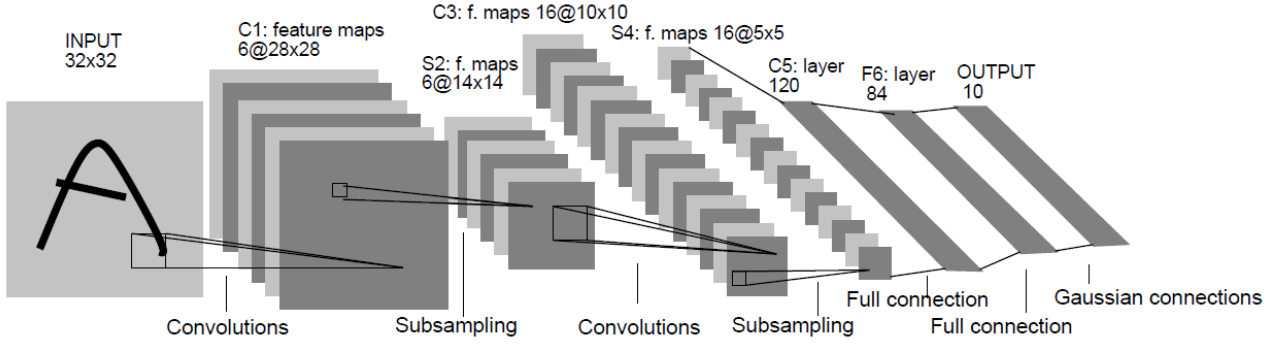


FIGURE 2 – LeNet-5 architecture with the hyperparameter configuration used in the original paper

Algorithm	Parameters
Grid search	$\emptyset$
Random search	$p = 5.4 \cdot 10^{-6}$
Particle Swarm Optimization	$S = 5, \phi_l = 2, \phi_g = 2, w = 0.5$
BOHB	$min_{epochs} = 10, max_{epochs} = 40, N_{iterations} = 20$

FIGURE 3 – Algorithm parameters

Unless indicated otherwise, the number of epochs for every trainings is set at 40. The probability parameter p of Random Search is chosen so as to consume a similar number of epochs as PSO and BOHB, which is around 4000 epochs per optimisation run.

For every method, we consider the following hyperparameter space to search :

Hyperparameter	Range
F6	[125,325]
Number of C1 filters	[1 :32]
Number of C3 filters	[1 :32]
Number of C5 filters	[40 :130]

FIGURE 4 – Hyperparameter space

We select this hyperparameter space by using the initial hyperparameter configuration used in LeNet-5 and explore neighbouring configurations to it.

Although the implemented algorithms enable increased efficiency compared to a babysitting or grid search approach, each optimization run took important amounts of time, allowing minimal repetitions to validate the results, leaving significant space to interpretation of the results and little quantitative results. We still present the results in the following section.

5 Results

Running the algorithms returns us the following results :

Due to the extreme resource consumption of grid search ($\sim 740\,000\,000$ epochs and $\sim 6\,000\,000$ compute hours), we weren't able to do a full run for this algorithm.

From these results, we observe that both PSO and BOHB provide good solutions for hyperparameter optimization, with significantly reduced costs compared to Grid Search or Random Search. From this experiment, there is no clear way of knowing which of BOHB or PSO is best to solve this specific hyperparameter problem (whether “best” is defined by convergence speed or reliability).

Algorithm	Hyperparameter config				Number of training epochs	Validation accuracy
	F6	C1	C3	C5		
PSO	324	26	19	40	$20 \times 5 \times 40$	66.8%
Genetic	324	26	19	40	$10 \times 8 \times 10$	66.8%
BOHB	318	30	15	44	40×40	66.5%
Grid Search	-	-	-	-	$200 \times 32 \times 32 \times 90 \times 40$	-
Random Search	224	27	30	40	$p \times 200 \times 32 \times 32 \times 90 \times 40$	67.83%

FIGURE 5 – Optimal hyperparameter configurations